Measuring Feedback-Driven Improvement in Lightweight Code-Security Verifiers for Spec-Driven Code Generation

PI: Chae (Kent State University at Stark, Computer Science) — SOLO, no Co-PI.

Cash funding needed: $80,000
AWS Promotional Credits needed: up to $40,000 (conditional, stretch tier — see Funds Needed)
Amazon contact (optional): —

Abstract

We build a lightweight verifier for the security of LLM-generated code and measure how it improves from false-positive and false-negative feedback. Spec-driven code generation is a primary production path, yet recent benchmarks report that even the strongest models rarely produce programs that are both secure and functionally correct, and existing benchmarks measure static snapshots, not whether verification improves over time. We build an oracle-guided corpus of LLM-generated programs: public-repository tasks and vulnerable-function instances in Python, run against CodeQL, Semgrep, and fuzzing harnesses, where oracle agreement is bounded evidence for labeled defects, not ground truth. On this corpus we fine-tune an open 30B-A3B model (LoRA) into a verifier that ranks candidate defects, and run a feedback loop in which oracle-resolved outcomes update detection thresholds and retraining batches, evaluated prequentially on data the loop never adapts to. A calibrated escalation policy routes low-confidence cases to simulated human review under budget. We report precision and recall against static baselines at matched cost, the improvement curve on unseen instances, and the cost-quality Pareto relative to general-purpose model judges. Deliverables: open verifier (MIT-licensed code and protocol), oracle-validated corpus under upstream terms, evaluation protocol, escalation benchmark. No runtime or cloud-protection claims: an offline code-level screening study.

Keywords: LLM-generated code security, spec-driven code generation, execution-oracle verification, fine-tuned security verifier, feedback-driven improvement, calibrated human escalation, secure code generation

Introduction

Spec-driven code generation — natural-language specifications compiled to code by large language models — is moving from research demo into generation pipelines. Published studies document security failures in generated code (∼40% of generated programs vulnerable [1]); benchmarks assess functionality and security in repository tasks (CodeSecEval [2]; RealSec-bench [3]); and security-hardening methods [4] show improvements under specific settings — these are three different kinds of evidence, not one trend. CodeSecEval already covers a Python, execution-validated, oracle-labeled corpus; the open gap is feedback evaluation, not corpus existence: none of these measures whether a verification layer improves as it accumulates feedback over time.

We test whether FP/FN-targeted scheduling improves a post-hoc verifier beyond standard SFT on identical candidate IDs at matched acquisition and adaptation budgets. A five-way, cluster-disjoint split separates training, feedback, calibration, non-adapting audit, and final holdout; its pre-registered primary contrast is oracle cost per correctly identified defect at a fixed recall floor. A lightweight LoRA verifier (open 30B-A3B MoE, 3B active; in-line target under 100 ms) couples to asynchronous oracle auditing and calibrated escalation to simulated human review under budget — an offline proxy, not a human study.

This proposal aligns with four items of the Amazon Security CFP [5], [6] — vulnerability detection systems that self-improve from false positive/negative feedback, "[p]ublic benchmarks for security in spec-driven code generation," capability-aware human-escalation recognition, and "AI-assisted secure code generation and security enhancement of existing codebases" — with the self-improving-detection item as the core alignment. Related adaptive mechanisms exist for general codebases (SIVA [7], IRIS [8], SSVD [9]) and SAST false-positive filtering via iterative re-prompting (Sifting the Noise [10]), but none is measured on spec-driven LLM-generated code with execution oracles and matched-budget controls. This is a code-level static screening study — no runtime or agentic-telemetry claims — distinct from our prior ARA submissions in domain, method, and data.

Methods

Oracle-guided corpus. Scope is frozen at M3: Python core, with JavaScript as a conditional stretch entered only if the Python path passes M4 and budget allows. We target a 2024 CWE Top-25 subset {79, 89, 918, 78, 94, 22, 798, 352, 502, 863}, selected for KEV presence and web/cloud exploit severity, all confirmed in the MITRE/CISA 2024 Top-25 [13]. Generation tasks: (i) public-repository task instances in the SWE-bench format [11] (Python repos, MIT-licensed); (ii) vulnerable-function instances built from disclosed CVEs in public Python repos — Big-Vul and PrimeVul [12] are C/C++ corpora and are excluded as corpus inputs; (iii) seeded-bug controls with known provenance. Each task type is a distinct instance unit; per-CWE$\times$split target sample sizes fixed at M1 in the correspondence table; all instances share one generator family (Qwen3-Coder), so generalization claims are scoped to that family absent a held-out generator.

Every artifact runs an oracle battery by CWE availability: CodeQL and Semgrep for all ten CWEs; fuzzing (Atheris / Jazzer.js [14]) where a differential execution trigger exists; logic-level CWEs (352, 863, 798) are static-only by pre-declaration — the specification is the only available oracle of the intended authorization boundary — with a static-oracle fallback if harness synthesis fails. An executable CWE-policy witness suffices for positive inclusion even if CodeQL and Semgrep are silent; two-tool agreement without a witness is weak evidence, never equivalent to witness validation. M3 fixes per-CWE witness-share minima and SAST-blind candidate selection; below-minimum or static-only classes remain scoped to oracle-conditioned screening. The negative class is two-tier: a primary control accepted as negative only when justified against the target CWE's declared policy under fixed fixture/environment assumptions — no-injection provenance alone is not treated as proof of security — and oracle-silent candidates (secondary, SAST-coverage-limited stratum, never pooled with the primary control). The pre-registered M4 gate requires deterministic reproduction for static-only CWEs and $\geq$95% two-run reproduction for fuzz-dependent CWEs, with per-cause family exclusion rather than a judgment call at M6.

Verifier. We fine-tune Qwen3-Coder-30B-A3B-Instruct (MoE, 3B active per call) with LoRA/QLoRA on local GPUs to output ranked candidate defects with CWE tag, code span, and calibrated confidence. The verifier is a synchronous in-line gate with a target — not an achieved guarantee — of under 100 ms per call, under a latency contract (token caps, batch size, quantization, warm vs. cold) fixed at the M1 cost pilot and measured end-to-end at p50/p95 on local hardware; if missed, we report the latency-quality frontier and narrow the in-line-gate claim rather than the offline feedback study. The oracle battery runs asynchronously as batch auditing, since the oracles that define the labels are too slow and environment-bound to serve as the gate. This differs from co-decoded security LLMs (CoSec [15]; CoSEFA [16]), which are not post-hoc and are not evaluated against oracle-labeled feedback at matched cost.

Self-improvement loop. For batch t, predictions are logged using only pre-t labels; labels are then revealed and scored; only afterward are the model, thresholds, and calibrator updated for $t{+}1$ — a non-adapting audit split, never touched by any update, is scored under the identical frozen rule. Oracle-resolved outcomes on the feedback split — never the final-evaluation split — update per-CWE detection thresholds, a rolling LoRA retraining batch with focal weighting on the failing class, and the escalation boundary τ. The single pre-registered primary contrast is total oracle-verification cost per correctly identified defect at a fixed recall floor, between FP/FN-targeted scheduling and a matched-budget standard-SFT control on the identical candidate IDs (not merely the same count), so only ordering/weighting differs; a separate random-feedback arm instead changes which candidates are acquired and is reported alongside with its oracle search cost. Minimum useful effect δ, maximum rounds/labels/compute, and the saturation rule are fixed at M3, before any loop run. Cost accounting separates oracle search/failures, labels, adaptation, inference, and simulated review; oracle-cost/defect is primary, end-to-end cost/defect reported under the same recall floor and declared amortization horizon. This differs from inference-time SAST re-prompting (Sifting the Noise [10]); we retrain the detector and report the cost at which improvement saturates against this pre-fixed rule.

Capability-aware escalation. Calibrated confidence is the model's estimated probability that its own predicted label (defect or no-defect) is correct, not the probability a defect exists; when it falls below a per-CWE threshold τ, a case routes to a simulated human-review stage. Routing is compared against random and uniform-threshold baselines at equal budget, with simulated reviewer accuracy varied jointly with the verifier's own error pattern — the two can share or offset mistakes — reviewer accuracy, service time, and capacity remain declared assumption variables with a sensitivity grid. High-confidence false negatives are a named, separately reported evaluation cell. This operationalizes the CFP's capability-aware item as an

offline proxy over ESCALATE [17], CalibCode [18], and HackerOne Code [19].

Novelty scope. LoRA SFT, focal loss, and calibrated abstention (Chow-style reject options) are each standard; the contribution is a falsifiable same-ID, matched-budget test of FP/FN-targeted scheduling against standard SFT, with a bounded-evidence labeling protocol, cost-quality Pareto against LLM judges, and budgeted escalation benchmark as secondary artifacts. We claim no new SFT algorithm, detector architecture, or first-of-kind corpus. Related work targets different mechanisms — SIVA [7] (prompt/memory adaptation), IRIS [8] (hybrid static analysis), SSVD [9] (pseudo-label learning), SamplingVD [20] (imbalance sampling), SPLVD [21] (self-paced sample selection), Sifting the Noise [10] (agentic FP triage), CodeSecEval [2] (pre-hoc generator-side; execution-guided refinement), MetaMorphSES [22] (generator-detector co-evolution) — and we do not claim an absence of execution oracles or matched-budget controls in that work. This post-hoc verifier test uses a frozen spec-driven generation distribution with explicit label-acquisition and adaptation budgets, compared against non-targeted schedules rather than a claim that execution feedback or adaptive detection is new.

Evaluation

Instances are partitioned by task-prompt family and repository cluster, strictly disjoint across splits (seed/time as secondary keys), into training, feedback (loop-only), calibration (τ-set), audit (non-adapting), and final-holdout; near-duplicates are removed. The split is frozen at M3, before any loop run, and the final-holdout's label-generation path is recorded independently of its non-exposure.

On final-holdout we report precision/recall/F1 per CWE and overall against CodeQL alone, Semgrep alone, their union, the frozen verifier, and a frontier LLM judge at matched compute, as cost per flagged defect. Matching is defect-level (ranked span plus CWE, pre-registered tolerance); security metrics are conditional on functionally valid, buildable code, with build/timeout outcomes entering a joint-outcome denominator. At fixed per-CWE alert budgets we report residual miss risk and positive yield; uncertainty uses per-seed intervals ($n \geq 3$) with cluster-preserving pairing, and SAST weak-label results appear in a separate column, never merged with witness-validated numbers.

The self-improvement curve is prequential F1 versus feedback round plus the non-adapting audit curve, per-seed intervals, with frozen-verifier, threshold-only, retraining-only, matched-budget standard-SFT, and random-feedback controls; F1 and saturation are secondary diagnostics of the primary cost/defect contrast at fixed recall, and the final-holdout opens once at M10. Escalation is reported as risk-coverage curves at fixed human budgets, learned-τ vs. uniform threshold; reviewer parameters remain declared simulation assumptions with sensitivity ranges.

Honesty rules: the no-injection control is a primary negative only when justified as above, oracle-silent code is a secondary stratum never pooled as benign; cross-language (JS) results are reported separately, never pooled with Python; oracle coverage is a declared boundary with no claim for unknown or oracle-invisible classes; observational and controlled results are reported separately, never pooled; oracle-unresolved instances are excluded from the conditional precision/recall/F1 denominator only, and are retained in coverage, attrition, and cost accounting as a funnel count. AWS credits, if awarded, fund only the large-scale fuzz/eval farm and multi-node stress runs (stretch) and gate no core deliverable.

Preliminary Evidence. An N=30 feasibility pilot on the vulnerable-function unit (Semgrep the sole SAST tool — CodeQL not run; all 30 instances generated by the committed generator Qwen3-Coder-30B-A3B-Instruct) measured: buildable yield 100% (30/30); Semgrep witness fraction 0% (0/30) under the standard `p/python` pack, whose security rules require web-framework request sources; two-run reproduction 100% (30/30). Manual inspection (diagnostic, not a witness) found the vulnerable pattern in 20/30 and injection-safe code in 10/30. With zero resolved labels, cost per resolved label is undefined; total pilot cost \$0.0022 ($\$\,7.3 \times 10^{-5}$ per attempt, stated API-equivalent rates); the per-CWE cluster SD of mean instance cost $\$\,2.7 \times 10^{-5}$ (n=6) is recorded alongside, not replacing, the planning assumption. In-line-gate p50/p95 latency remains an M1 cost-pilot item; the null witness result motivates the repository-context and seeded-bug units.

Impact

We test targeted feedback's oracle-cost benefit and report end-to-end cost separately, publishing negative or inconclusive curves under the pre-registered rule. The open verifier, corpus, protocol, and escalation benchmark remain reusable at the input-schema level (no pipeline-compatibility claim). AWS product mapping follows the Budget document; training resources requested are limited to credits. Independent studies of

AWS's own code assistants report the same residual-vulnerability class this project screens for [23], grounding the Amazon relevance claim.

Expected Results

Timeline & Milestones

Quarter	Milestones	Deliverable
Q1 (M1-3)	Correspondence table + pilot cost measurement; three-oracle battery + harness-synthesis sub-milestone (500 pilot instances); **M3**: split frozen	Oracle pipeline + labeling protocol v1 + frozen split manifest
Q2 (M4-6)	**M4** per-CWE gate; corpus to scale; LoRA verifier v1 + frozen baseline; **M5** pilot loop; **M6** independent effect gate (pre-main-loop)	Go/no-go per pre-defined rule (M6)
Q3 (M7-9)	Main feedback loop; ablation arms (threshold-only / retraining-only / standard-SFT / random-feedback); escalation τ calibration	Self-improvement curve (interim) + ablation table
Q4 (M10-12)	Final-holdout evaluation (single open, no reselection); cost-quality Pareto incl. frontier judge; escalation benchmark; manuscript	**M12**: full OSS release + Zenodo DOI + paper submission

M3 separates pilot-audit from confirmatory audit clusters; M6 reads only the former, and neither gate-driven reselection nor retuning may use the latter or final holdout; the pre-declared minimum effect size δ separates a useful gain, an excluded gain, and an inconclusive result, and a 95% paired interval of the matched-budget difference that includes zero is treated as inconclusive at the current sample size, not as proof of no effect. If the matched-budget contrast is inconclusive, the public protocol — labeling protocol, frozen split manifest, adapters, safe fixtures, and the negative curve — stands as the deliverable.

Open Source & Reproducibility

We release `spec-verify` under a separated-rights policy: code, protocol, and harnesses under MIT; the oracle-validated corpus under upstream terms (SWE-bench-format tasks: MIT, verified; CVE-sourced and seeded instances: our generated artifacts MIT-licensed, upstream code, issue text, and CVE patches under their own licenses, recorded per-instance in the correspondence table); base-model weights are not redistributed — only LoRA adapters plus a pointer. The minimum public artifact — frozen split manifest, oracle battery configuration, labeling protocol, evaluation harness, escalation benchmark, safe local fixtures, LoRA adapters — reproduces every headline number on the released safe-fixture subset; full-corpus reproduction additionally requires the documented legal retrieval recipe (source, version, hash) for upstream-licensed material not safe to redistribute. Vulnerable instances carry sanitized witnesses: a local fixture in which the policy violation is actually observed, without external damage or live exploit payloads; inert substitutes alone are not a security witness. Release: M6 alpha (corpus subset + verifier v1), M12 full release with Zenodo DOI; no personal data or secrets in public artifacts.

Funds Needed

Item	USD
Graduate Student, hourly (1,500 hrs @ $20/hr)	$30,000
Undergraduate Student, hourly (800 hrs @ $15/hr)	$12,000
PI effort (12-month continuous)	$25,000
Travel (2 conferences)	$5,000
Equipment / storage / oracle infra (sandboxes, CPU fuzz instances)	$4,000
External inference — frontier LLM judge baseline (API)	$3,000
Contingency	$1,000
Total	**$80,000**

Single-institution (Kent) solo budget: one graduate student, one undergraduate student, and PI effort; no Co-PI, no sub-award. Student cost is hourly wages only — tuition, fringe, and fully-loaded treatment are pending institutional confirmation and excluded from the cash total. **AWS credits ($40,000, conditional stretch tier)** fund SageMaker, EC2 (CPU and GPU), S3, and CloudWatch per the Budget document (AWS Public Dataset usage: N/A); credits are stretch-only (every primary deliverable is locally reproducible); core/stretch counts and prices are fixed at M1.

Appendix A — References

(does not count toward the 4-page limit; limit two pages)

[1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *Proc. 2022 IEEE Symp. Security and Privacy (SP)*, May 2022, pp. 754–768, doi: 10.1109/SP46214.2022.9833571.

[2] J. Wang et al., "Toward Secure Code Generation: Bridging Correctness and Security via Task-Adaptive Vulnerability Modeling and Execution-Based Benchmarking," *Proc. ACM Softw. Eng.*, vol. 3, no. ISSTA, Art. ISSTA068, Oct. 2026, doi: 10.1145/3832159.

[3] Y. Wang, Z. Zhang, C. Wang, X. Xu, M. Liu, Y. Wang, J. Chen, and Z. Zheng, "RealSec-bench: A Benchmark for Evaluating Secure Code Generation in Real-World Repositories," in *Findings Assoc. Comput. Linguistics: ACL 2026*, Jul. 2026, pp. 35866–35883, doi: 10.18653/v1/2026.findings-acl.1786.

[4] J. He and M. Vechev, "Large Language Models for Code: Security Hardening and Adversarial Testing," in *Proc. 2023 ACM SIGSAC Conf. Computer and Communications Security (CCS '23)*, Nov. 2023, pp. 1865–1879, doi: 10.1145/3576915.3623175.

[5] "Amazon Security Call for Proposals, Fall 2026," Amazon Research Awards. Accessed: Oct. 1, 2026. [Online]. Available: https://amazon.science/research-awards/call-for-proposals/amazon-security-call-for-proposals-fall-2026

[6] "Frequently Asked Questions," Amazon Research Awards. Accessed: Oct. 27, 2026. [Online]. Available: https://amazon.science/research-awards/frequently-asked-questions

[7] V. Walischewski, G. Zizzo, and K. Webster, "SIVA: Self-Improving Vulnerability Agent," presented at the NeurIPS 2025 Workshop on Reliable ML from Unreliable Data, Dec. 2025. [Online]. Available: https://research.ibm.com/publications/siva-self-improving-vulnerability-agent

[8] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in *Proc. Int. Conf. Learning Representations (ICLR 2025)*, 2025. [Online]. Available: https://iclr.cc/virtual/2025/poster/30698

[9] X. Yu, G. Lin, X. Hu, J. W. Keung, and X. Xia, "Less Is More: Unlocking Semi-Supervised Deep Learning for Vulnerability Detection," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 3, Art. 62, Feb. 2025, doi: 10.1145/3699602.

[10] Y. Xiong and T. Zhang, "Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering," *Proc. ACM Softw. Eng.*, vol. 3, no. ISSTA, Art. ISSTA009, pp. 197–220, Oct. 2026, doi: 10.1145/3832100.

[11] "SWE-bench," GitHub repository, Princeton NLP. Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/princeton-nlp/SWE-bench (MIT license, LICENSE.md, directly verified)

[12] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability Detection with Code Language Models: How Far Are We?" in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE 2025)*, May 2025, doi: 10.1109/ICSE55347.2025.00038. (PrimeVul; C/C++ benchmark, excluded as a corpus input in this proposal, CWE-family reference only.)

[13] "CWE-918: Server-Side Request Forgery (SSRF)," MITRE Common Weakness Enumeration; ranked #19 in the 2024 CWE Top 25 Most Dangerous Software Weaknesses. Accessed: Oct. 29, 2025. [Online]. Available: https://cwe.mitre.org/data/definitions/918.html

[14] "OSS-Fuzz: Continuous Fuzzing for Open Source Software," Google. [Online]. Available: https://oss-fuzz.com (Python/Atheris and JS/Jazzer.js harness guides)

[15] D. Li, M. Yan, Y. Zhang, Z. Liu, C. Liu, X. Zhang, T. Chen, and D. Lo, "CoSec: On-the-Fly Security Hardening of Code LLMs via Supervised Co-decoding," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA 2024)*, Sept. 2024, doi: 10.1145/3650212.3680371.

[16] X. He, D. Li, H. Wen, Y. Zhu, C. Liu, M. Yan, and H. Zhang, "CoSEFA: An LLM-Based Programming Assistant for Secure Code Generation via Supervised Co-Decoding," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE Companion '25)*, Jun. 2025, pp. 1198–1202, doi: 10.1145/3696630.3728609.

[17] J. Jung, F. Brahman, and Y. Choi, "Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement," in *Proc. Int. Conf. Learning Representations (ICLR 2025)* (Oral), 2025. [Online]. Available: https://iclr.cc/virtual/2025/oral/31838

[18] C. Spiess, D. Gros, K. S. Pai, M. Pradel, M. R. I. Rabin, A. Alipour, S. Jha, P. Devanbu, and T. Ahmed, "Calibration and Correctness of Language Models for Code," in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE 2025)*, Apr. 2025, doi: 10.1109/ICSE55347.2025.00040.

[19] "HackerOne Code," HackerOne. [Online]. Available: https://www.hackerone.com/product/code

[20] X. Yang, S. Wang, Y. Li, et al., "Does Data Sampling Improve Deep Learning-Based Vulnerability Detection? Yeas! and Nays!" in *Proc. 45th IEEE/ACM Int. Conf. Software Engineering (ICSE 2023)*, May 2023, pp. 2287–2298, doi: 10.1109/ICSE48619.2023.00192.

[21] Z. Cheng, Y. Yang, H. Zhang, L. Yang, J. Hu, J. Xu, B. Liu, and H. Shen, "Leveraging Self-Paced Learning for Software Vulnerability Detection," *ACM Trans. Softw. Eng. Methodol.*, Aug. 2026, doi: 10.1145/3844141.

[22] M. R. Hasan and H. Bagheri, "Self-Discovering Security Oracles: Meta-learning Vulnerability Detection Strategies through Adversarial Self-Play," in *Proc. 41st IEEE/ACM Int. Conf. Automated Software Engineering (ASE 2026)*, Oct. 2026, doi: 10.1145/3832783.3837512.

[23] W. Cheng, K. Sun, X. Zhang, and W. Wang, "Security Attacks on LLM-based Code Completion Tools," *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 22, pp. 23669–23677, Apr. 2025, doi: 10.1609/aaai.v39i22.34537.

Appendix B — Previously Funded Project Summary

(does not count toward the 4-page limit; official 2026-10-01 ARA template places the previously-funded summary at Appendix B when the CV is a separate document — see the accompanying CV document)

No previously funded project. The PI (Chae) has not received cash funding or AWS Promotional Credits, directly or indirectly, from Amazon Research Awards or any other Amazon program within the past 3 years, and has no previously funded Amazon Research Award project to report. Confirmed by the PI, 2026-10-29.